

Technical Assessment: Real-Time Signal Generation & Backtesting Service

Overview

Build a full-stack application that ingests historical market price data, generates trading signals from a predictive model you build, exposes the system through a concurrent FastAPI backend, and surfaces results through a Next.js dashboard with live updates. The core of the assessment is twofold: (1) a backend that processes many instruments **concurrently** under a measurable latency target, and (2) a model whose quality you must **rigorously evaluate against a baseline**, not merely run.

This problem deliberately sits in the financial time-series domain so that your existing experience applies directly. We are not asking you to learn an unfamiliar field over a weekend — we are asking you to demonstrate the depth behind the systems you have already built.

Business Context

Scenario: A desk needs a service that, given recent price history for a set of instruments, produces a directional signal (up / down / flat) for the next period along with a confidence value — and, critically, a way to judge whether those signals are actually any good before risking capital on them. Speed matters because signals for many instruments must be served on demand without the request blocking.

What we care about most: not that the model is profitable, but that you can build a concurrent service correctly, measure its performance honestly, and prove your model beats a trivial baseline using sound evaluation methodology.

Part 1 — Predictive Model & Honest Evaluation

This is the graded core of the assessment. A working API around a model you cannot defend is worth little.

Model

- Build a model that, given a window of recent OHLC (or close-price) history for an instrument, predicts the **direction** of the next period (up / down / flat) with a confidence score.
- You choose the approach (classical ML, a small neural net, gradient boosting, statistical method — your call). **You must justify the choice in writing**, including why it suits this data and what its limitations are.
- You must clearly state, in your own words, what the model is and is not. (e.g. if you use a neural network, what kind; if you use an autoML or hyperparameter-search tool, what it does and what it does *not* do.) Sloppy or incorrect terminology will count against you.

Evaluation — Mandatory

You must produce an honest evaluation of your model. This section is **not optional** and is weighted heavily.

- Implement a **proper train/test split that respects time order** (no look-ahead / no leakage — explain how you prevented it).
- Compare your model against at least **two baselines**, one of which must be trivial (e.g. "always predict up", "predict last direction persists", or a random predictor). A model that cannot beat a coin flip should be reported as such.
- Report a defined set of metrics and **define each one in your writeup in your own words**: at minimum **directional accuracy**, and at least two of {precision/recall or F1, a simple backtested P&L or cumulative return, Sharpe ratio, maximum drawdown}.
- State plainly whether your model is actually better than the baselines, and by how much. "It works" is not an acceptable conclusion — show the numbers.

Provide your sample data with the submission (a few liquid instruments is enough — forex, indices, or equities). If you generate or download data, include the script or the files.

Part 2 — Concurrent FastAPI Backend

API Endpoints

1. **POST /instruments/load** — register one or more instruments and their historical data for use by the service.

2. **POST /signals/generate** — accept a list of instrument identifiers and return a signal for **each**. The service must process the instruments **concurrently**, not one-by-one in a blocking loop.
3. **GET /signals/{job_id}** — retrieve results for a generation job (use this if you choose a background/job model for larger batches).
4. **GET /health** — basic health/readiness check.

Technical Requirements

- Use **FastAPI**.
- Signal generation across multiple instruments must run **concurrently**. You decide the mechanism (asyncio concurrency, a task/worker model, a process or thread pool for CPU-bound inference) but you must **explain in your writeup why your chosen mechanism is correct for the kind of work being done** — specifically, whether the per-instrument work is I/O-bound or CPU-bound, and why that determines the right concurrency primitive. This explanation is graded.
- Handle per-instrument failures gracefully — if one instrument errors, the others must still return.
- All responses in JSON.
- Input validation on all endpoints (unknown instrument, malformed payload, empty batch).

Performance — You Must Measure It

- Include a **load/benchmark script** (Python, k6, locust, or similar) that fires concurrent requests and reports throughput and latency percentiles (p50 / p95 / p99).
- Report your numbers in the README, with the hardware you ran on and the concurrency level. We want the *actual* measured figures, not a target.
- Demonstrate, with numbers, that concurrent generation is faster than a naive sequential loop for a batch (e.g. 20+ instruments). Showing the before/after is the point.

Example /signals/generate response

```
{
  "job_id": "job_8f2a",
  "status": "completed",
  "generated_at": "2025-06-07T10:30:00Z",
  "batch_latency_ms": 412,
  "signals": [
    {
      "instrument": "EURUSD",
      "direction": "up",
```

```
    "confidence": 0.71,  
    "model_version": "v1",  
    "status": "ok"  
  },  
  {  
    "instrument": "BADSYM",  
    "status": "error",  
    "error": "instrument not loaded"  
  }  
]  
}
```

Part 3 — Next.js Dashboard

Build a frontend that consumes your backend. This verifies the full-stack and real-time charting experience described on your CV.

- **Next.js** (App Router or Pages Router), responsive (mobile + desktop).
- An instrument selection / batch view where the user triggers signal generation and sees per-instrument results, including the error state for failed instruments.
- A **price chart** for a selected instrument that displays the OHLC/close history and marks the generated signal on it. You may use a charting library or build a custom one — state which and why.
- A **live-updating element**: stream generation progress or push updated signals to the dashboard as they complete, using **WebSockets** (or SSE — justify the choice). Polling alone does not satisfy this requirement.
- Loading and error states surfaced clearly to the user.

Deliverables

1. GitHub Repository (public, or grant access) containing:

- Complete FastAPI backend code
- Complete Next.js frontend code
- Model training + evaluation code, and the sample data (or a script to fetch/generate it)
- The load/benchmark script
- **README.md** with:

- Step-by-step setup instructions (a reviewer must be able to run it from your README alone)
 - API documentation with example requests/responses
 - Your measured performance numbers (p50/p95/p99, hardware, concurrency level, sequential-vs-concurrent comparison)
 - Your model evaluation results and baseline comparison
 - A **design-decisions writeup (max 600 words)** covering: model choice and what it is/isn't, your concurrency mechanism and why it fits I/O- vs CPU-bound work, how you prevented look-ahead in evaluation, and your WebSocket/SSE choice
- requirements.txt and package.json, plus .env.example if applicable

2. Demo & Walkthrough Video (5–8 minutes) — required:

- Show the running application end to end: load instruments, generate signals for a batch, see them appear live on the dashboard with the chart.
- Run your benchmark script live and talk through the numbers.
- Walk through your own code and explain, in your own words: how concurrency is achieved, where the model runs, and how the evaluation prevents leakage. Treat this as if you are explaining the system to a teammate who will maintain it.

3. Sample Data — included in the repo, enough to run the full pipeline without external setup where possible.

Minimum Viable Submission (Must Have)

- Model generates directional signals with a confidence value.
- Evaluation present: time-respecting split, at least two baselines (one trivial), defined metrics with stated numbers, and an explicit better/worse-than-baseline conclusion.
- FastAPI backend with the required endpoints, processing a batch of instruments concurrently, with graceful per-instrument failure handling.
- Benchmark script that produces real latency/throughput numbers, reported in the README.
- Next.js dashboard that triggers generation, shows per-instrument results, renders a price chart, and updates live via WebSocket/SSE.
- README with working setup instructions and the design-decisions writeup.

- Walkthrough video.
- Runs locally without errors.

Nice to Have

- Background-job model for large batches with a queryable job status.
- Model versioning, or comparison of more than one model approach.
- Confidence calibration discussion (is a 0.7 confidence actually right ~70% of the time?).
- Caching of recent results, and a measured demonstration of its effect.
- Containerised setup (Docker / docker-compose).
- Optional LLM component — e.g. a natural-language summary of why a signal was generated, or a query interface over the results. If you add this, treat it as a clearly-scoped extra, not a substitute for the ML/eval core, and describe what the LLM is doing.
- Deployed live version.

Timeline

Complete and submit within 48 hours of receiving this assessment (intended for a weekend). Submit whatever you have at the deadline — a smaller, honest, well-explained submission beats an over-claimed one.

Notes

- **Honesty over polish.** We would rather see "my model barely beats the persistence baseline and here is why" than an unsupported claim of strong performance. Overstated metrics without methodology will count against you more than a modest, well-evidenced result.
- **Use your own judgment on tools** where the spec does not mandate one — and be ready to defend each choice in the writeup and video.
- **It must be your code.** The walkthrough video should let you explain any part of the system on the spot.

- **Questions:** if anything about scope, data, or expected edge-case behaviour is unclear, reach out before you start building.
-

Good luck!